

Especificación de Requisitos de Software (ERS)

Sistema de Gestión Cívica de Incidentes Viales
ReportaTuCalle



Elaborado por:

Bryan Aldrin Chontay Campos
Paul Fran León Gonzales
Arturo Miguel Osorio Miranda
Erick Joel Sánchez Longa
Joaquín Marcelo Torres Reátegui

Proyecto *ReportaTuCalle*

12 de julio de 2026

Índice general

1. Introducción	2
1.1. Propósito	2
1.2. Alcance	2
1.3. Definiciones, Siglas y Abreviaturas	2
1.4. Descripción General del Sistema	3
1.5. Usuarios del Sistema	3
2. Requisitos Funcionales	4
2.1. Módulo de Autenticación y Cuentas	4
2.2. Módulo de Usuarios y Perfil Cívico	4
2.3. Módulo de Categorías de Incidentes	5
2.4. Módulo de Reportes de Incidentes	5
2.5. Módulo de Media (Imágenes)	6
2.6. Módulo de Notificaciones en Tiempo Real	6
2.7. Módulo de Optimización de Rutas	6
2.8. Módulo de Documentación de la API	7
3. Requisitos No Funcionales	8
3.1. Atributos de Calidad	8
3.2. Restricciones Técnicas	8

1. Introducción

1.1. Propósito

El presente documento tiene como propósito describir de manera clara y organizada los requisitos de software del sistema *ReportaTuCalle*, una plataforma cívica que permite a los ciudadanos reportar incidentes viales (baches, fugas de agua, semáforos dañados, acumulación de basura, entre otros) georreferenciados sobre un mapa interactivo, y a los supervisores y administradores gestionar, priorizar y optimizar la atención de dichos reportes.

Este documento sirve como referencia para el equipo de desarrollo, el equipo de control de calidad (QA) y cualquier persona interesada en conocer qué funcionalidades ofrece el sistema actualmente implementado.

1.2. Alcance

El sistema *ReportaTuCalle* está compuesto por:

- Un **backend** desarrollado en Java 21 con Spring Boot 3, organizado en los módulos: autenticación, usuarios, reportes, categorías, medios (imágenes), notificaciones y optimización de rutas.
- Un **frontend** tipo aplicación web (SPA) desarrollado en React 18, con mapa interactivo basado en Leaflet.
- Una **base de datos** relacional PostgreSQL con la extensión geoespacial PostGIS.

El sistema contempla tres tipos de usuario (roles): **Ciudadano**, **Supervisor** y **Administrador**, cada uno con funcionalidades específicas descritas en el presente documento.

1.3. Definiciones, Siglas y Abreviaturas

- **RF**: Requisito Funcional.
- **RNF**: Requisito No Funcional.
- **JWT**: *JSON Web Token*, estándar utilizado para la autenticación de usuarios.
- **SSE**: *Server-Sent Events*, mecanismo utilizado para notificaciones en tiempo real.

- **TSP:** *Travelling Salesman Problem*, problema del agente viajero, usado para optimizar rutas.
- **MST:** *Minimum Spanning Tree*, árbol de expansión mínima.
- **PostGIS:** extensión espacial de PostgreSQL para el manejo de datos geográficos.
- **CITIZEN, SUPERVISOR, ADMIN:** roles de usuario definidos dentro del sistema.

1.4. Descripción General del Sistema

ReportaTuCalle busca cerrar la brecha entre los ciudadanos que detectan problemas en la vía pública y las entidades responsables de resolverlos. A través de un mapa interactivo, cualquier ciudadano registrado puede reportar un incidente indicando su ubicación exacta, una descripción, una fotografía y una categoría. Los supervisores reciben estos reportes, pueden asignárselos a sí mismos o recibir asignaciones, actualizar su estado y utilizar un motor de optimización de rutas para atender varios incidentes de forma eficiente. Los administradores gestionan las categorías del sistema y los roles de las cuentas de usuario.

1.5. Usuarios del Sistema

Rol	Descripción
Ciudadano (CITIZEN)	Usuario que reporta incidentes, respalda (<i>endorsa</i>) reportes de otros ciudadanos, visualiza el mapa de incidentes cercanos y acumula un puntaje cívico (gamificación).
Supervisor (SUPERVISOR)	Usuario encargado de atender los reportes: puede auto-asignarse reportes, actualizar su estado y generar rutas óptimas de atención.
Administrador (ADMIN)	Usuario con permisos totales: gestiona categorías, cuentas de usuario y roles, y puede asignar reportes de forma masiva a los supervisores.

2. Requisitos Funcionales

A continuación se listan los requisitos funcionales (RF) actualmente implementados en el sistema, agrupados por módulo funcional.

2.1. Módulo de Autenticación y Cuentas

- RF-01** El sistema debe permitir el registro de una nueva cuenta de ciudadano mediante correo electrónico y contraseña.
- RF-02** El sistema debe permitir el inicio de sesión (*login*) de un usuario registrado, devolviendo un token de acceso (JWT) válido.
- RF-03** El sistema debe validar el token JWT en cada petición a un recurso protegido, rechazando las solicitudes con token inválido, expirado o ausente.
- RF-04** El sistema debe restringir el acceso a los recursos según el rol del usuario autenticado (CITIZEN, SUPERVISOR, ADMIN).
- RF-05** El sistema debe permitir al administrador consultar el listado de todas las cuentas registradas.
- RF-06** El sistema debe permitir al administrador modificar el rol asignado a una cuenta de usuario.

2.2. Módulo de Usuarios y Perfil Cívico

- RF-07** El sistema debe permitir a un usuario autenticado consultar los datos de su propio perfil (nombre, apellido, teléfono, puntaje cívico).
- RF-08** El sistema debe permitir a un usuario autenticado actualizar los datos de su propio perfil.
- RF-09** El sistema debe incrementar automáticamente el puntaje cívico (*civic score*) de un ciudadano al realizar acciones relevantes (por ejemplo, crear o respaldar un reporte).
- RF-10** El sistema debe exponer un listado (*leaderboard*) de los ciudadanos ordenado por puntaje cívico.

2.3. Módulo de Categorías de Incidentes

- RF-11** El sistema debe permitir consultar el listado completo de categorías de incidentes disponibles (baches, fugas de agua, semáforos dañados, acumulación de basura, entre otras).
- RF-12** El sistema debe permitir consultar el detalle de una categoría específica.
- RF-13** El sistema debe permitir al administrador crear una nueva categoría, definiendo nombre, descripción, color identificador en el mapa y tipo de algoritmo de optimización asociado.
- RF-14** El sistema debe permitir al administrador actualizar una categoría existente.
- RF-15** El sistema debe permitir al administrador eliminar (o desactivar) una categoría existente.

2.4. Módulo de Reportes de Incidentes

- RF-16** El sistema debe permitir a un ciudadano crear un nuevo reporte de incidente, indicando título, descripción, categoría, ubicación geográfica (latitud/longitud) y una fotografía de evidencia.
- RF-17** El sistema debe validar la información del reporte antes de almacenarlo (ubicación válida dentro del área de cobertura, ausencia de lenguaje ofensivo en la descripción).
- RF-18** Todo reporte creado debe iniciar en el estado **PENDING** (pendiente).
- RF-19** El sistema debe permitir consultar el listado general de reportes.
- RF-20** El sistema debe permitir consultar el detalle de un reporte específico por su identificador.
- RF-21** El sistema debe permitir actualizar los datos de un reporte existente.
- RF-22** El sistema debe permitir eliminar un reporte existente.
- RF-23** El sistema debe permitir consultar los reportes cercanos a una ubicación geográfica dada (búsqueda por proximidad).
- RF-24** El sistema debe permitir a un supervisor consultar el listado de reportes que tiene asignados.
- RF-25** El sistema debe permitir actualizar el estado de un reporte (**PENDING**, **ASSIGNED**, **IN_PROGRESS**, **RESOLVED**, **REJECTED**), respetando las transiciones válidas según el estado actual.
- RF-26** El sistema debe permitir asignar manualmente un reporte a un supervisor específico.

- RF-27** El sistema debe permitir a un supervisor auto-asignarse un reporte disponible.
- RF-28** El sistema debe permitir al administrador realizar la asignación masiva (*bulk-assign*) de varios reportes a uno o varios supervisores.
- RF-29** El sistema debe permitir a un ciudadano respaldar (*endorsar*) un reporte ya existente creado por otro ciudadano, incrementando su contador de respaldos.
- RF-30** El sistema debe impedir que un mismo ciudadano respalde más de una vez el mismo reporte.
- RF-31** El reporte debe permitir adjuntar una imagen de resolución (evidencia fotográfica) al marcarse como **RESOLVED**.

2.5. Módulo de Media (Imágenes)

- RF-32** El sistema debe permitir la subida de imágenes de evidencia asociadas a un reporte.
- RF-33** El sistema debe almacenar las imágenes utilizando un proveedor externo en la nube (Cloudinary) o, alternativamente, un almacenamiento local configurable.

2.6. Módulo de Notificaciones en Tiempo Real

- RF-34** El sistema debe notificar en tiempo real a los ciudadanos suscritos cuando se crea un nuevo reporte cercano (evento **REPORT_CREATED**) mediante un canal de eventos del servidor (SSE).
- RF-35** El sistema debe notificar en tiempo real al panel de administración cuando un reporte cambia de estado (evento **REPORT_UPDATED**) o es asignado (evento **REPORT_ASSIGNED**).
- RF-36** El sistema debe soportar el envío de notificaciones a través de un canal de correo electrónico configurable.

2.7. Módulo de Optimización de Rutas

- RF-37** El sistema debe calcular una ruta óptima de atención para un conjunto de reportes, seleccionando el algoritmo según el tipo asociado a la categoría del incidente: ruteo tipo agente viajero (**ROUTING**), flujo máximo en red (**FLOW**) o árbol de expansión mínima (**CONNECTIVITY**).
- RF-38** El sistema debe permitir a un supervisor solicitar la optimización de ruta indicando un punto de partida y un conjunto de destinos (reportes a atender).

- RF-39** El sistema debe permitir guardar el historial de una ruta calculada, incluyendo la distancia total recorrida y su estado.
- RF-40** El sistema debe permitir consultar el historial de rutas generadas previamente.
- RF-41** El sistema debe permitir actualizar el estado de una ruta guardada.

2.8. Módulo de Documentación de la API

- RF-42** El sistema debe exponer una documentación interactiva de todos los recursos de la API REST (OpenAPI / Swagger), accesible para los desarrolladores.

3. Requisitos No Funcionales

3.1. Atributos de Calidad

Atributo	Descripción
Mantenibilidad	El dominio de negocio debe mantenerse independiente de los detalles técnicos de infraestructura (framework, base de datos, HTTP), permitiendo modificar la tecnología sin alterar las reglas de negocio.
Extensibilidad	Debe ser posible incorporar nuevos algoritmos de optimización de rutas o nuevos canales de notificación sin modificar el código existente.
Bajo acoplamiento	Cada módulo de negocio (autenticación, usuarios, reportes, categorías, medios, notificaciones, optimización) debe mantenerse independiente, con su propio esquema de base de datos.
Disponibilidad de datos en tiempo real	Los cambios de estado de los reportes deben reflejarse de inmediato en los tableros de los usuarios conectados, sin necesidad de recargar la página.
Escalabilidad geoespacial	Las consultas de proximidad y agrupamiento de reportes en el mapa deben resolverse de forma eficiente incluso con un alto volumen de datos.
Testabilidad	La lógica de negocio debe poder probarse de forma unitaria sin necesidad de levantar la base de datos ni el servidor completo.

3.2. Restricciones Técnicas

- El backend debe ejecutarse sobre Java 21 y Spring Boot 3.5.x.
- La persistencia debe implementarse en PostgreSQL 16 con la extensión PostGIS 3.4 para el manejo de coordenadas geográficas.
- Las migraciones de esquema de base de datos deben ser versionadas y reproducibles (Flyway).

- El almacenamiento de imágenes debe soportar un proveedor en la nube intercambiable por almacenamiento local.
- El frontend debe implementarse como una aplicación de página única (SPA) en React, compatible con navegadores modernos.